

UNIVERSITY OF LORALAI

Department of Computer Science

ASSIGNMENT NO. 4

StudentRecord Class — Secure Academic Record Management

Field	Details
Student Name	Muhammad Naveed
Class ID	1176
Program	BS Computer Science
Course Area	Object-Oriented Programming
Submission Date	September 28, 2026

1. Problem Understanding

The assignment asks for a secure `StudentRecord` class that protects student academic information and prevents invalid or logically inconsistent object states. The required concepts include classes and objects, constructors, encapsulation, private data, setters/getters, validation, cross-field validation, and analytical testing. The assignment specifically states that the solution must not be a single `main()` program and that meaningful class functionality is compulsory.

2. Design Overview

Component	Purpose
Private data members	Prevent direct modification from <code>main()</code> .
Default constructor	Creates a safe initial object.
Parameterized constructor	Validates all six values during object creation.
Setters	Validate proposed changes and preserve old values on rejection.
Getters	Safely read private values.
Atomic update	Updates registered credits, earned credits and CGPA together.
Scholarship evaluator	Calculates eligibility from the required conditions.
Display function	Shows complete record and scholarship status.

3. Private Data Members

Data Member	Type	Validation / Role
<code>studentName</code>	string	At least 3 characters.
<code>rollNumber</code>	string	At least 4 characters; assigned only once.
<code>registeredCreditHours</code>	int	0–160; cannot be below earned credits.
<code>earnedCreditHours</code>	int	0–160; cannot exceed registered credits.
<code>cgpa</code>	double	0.00–4.00.
<code>disciplinaryCases</code>	int	0–3.

The assignment requires all six data members to be private and prohibits direct data access from `main()`.

4. Validation and State Protection

The implementation rejects invalid setter input instead of clamping or silently converting it. For example, CGPA 4.7 is rejected rather than converted to 4.0. Cross-field rules are also enforced: earned credits cannot exceed registered credits, and registered credits cannot be reduced below already-earned credits.

Roll-number assignment is state-dependent: a valid roll number can be assigned when the current value is "Not Assigned", but later attempts to replace it are rejected.

5. Constructor Strategy

The parameterized constructor validates registered credit hours before earned credit hours because the earned-credit rule depends on the final validated registered-credit value. Invalid values are replaced with the safe values specified by the assignment.

6. Atomic Academic Update

`updateAcademicProgress()` follows a transaction-style approach: it validates all proposed values first and commits the three changes only if every condition passes. This prevents partial updates, which the assignment explicitly identifies as unsafe.

7. Scholarship Logic

A student is eligible only when CGPA is at least 3.50, earned credits are at least 30, disciplinary cases are zero, and earned credits are at least 75% of registered credits. The code checks for zero registered credits before calculating the percentage.

8. Required Objects

Object	Purpose	Important values
Object 1	Default constructor + setters	Starts safely, then receives valid values.
Object 2	Valid parameterized object	Sara Ahmed, BSCS-202, 120/100, CGPA 3.70, cases 0.
Object 3	Invalid parameterized object	Invalid name/roll/credits/CGPA/cases; safe state demonstrated.
Object 4	Boundary-value object	160 registered, 160 earned, CGPA 4.00, cases 3.

9. Test-Case Evidence

#	Test	Expected	Actual
1	Empty name	Rejected	Rejected
2	Two-character name	Rejected	Rejected
3	Valid name after invalid	Accepted	Accepted
4	Assign roll BSCS-505	Accepted	Accepted
5	Change assigned roll	Rejected	Rejected
6	CGPA -0.01	Rejected	Rejected

#	Test	Expected	Actual
7	CGPA 4.01	Rejected	Rejected
8	CGPA 0.00	Accepted	Accepted
9	CGPA 4.00	Accepted	Accepted
10	Earned 150 when registered 100	Rejected	Rejected
11	Registered 60 when earned 80	Rejected	Rejected
12	Disciplinary -1	Rejected	Rejected
13	Scholarship with registered 0	Not Eligible	Not Eligible
14	Atomic 120, 90, 3.80	Accepted	Accepted
15	Atomic 120, 125, 3.50	Rejected; unchanged	Rejected; unchanged

The assignment requires all listed analytical tests to be demonstrated with clear labels.

10. Analytical Questions

Q1. Why should the data members be private?

Private members implement encapsulation and data hiding. If CGPA or earned credits were public, `main()` could directly insert invalid values and bypass the class validation rules.

Q2. Why validate in the parameterized constructor if setters validate?

The constructor is responsible for creating the first valid state. It must validate incoming values before storing them so an invalid object is never created.

Q3. Registered = 50, Earned = 70. What should happen?

Registered credits become 50. Earned credits become 0 because 70 is greater than the final validated registered-credit value. This preserves the cross-field rule.

Q4. Why reject `setRegisteredCreditHours(60)` when `registered = 100` and `earned = 80`?

The change would make earned credits 80 greater than registered credits 60. Therefore the update must be rejected and the previous valid registered value must remain.

Q5. Difference between `student.cgpa = 5.2` and `student.setCgpa(5.2)`?

Direct assignment would bypass validation, so it is prohibited by private access. `setCgpa(5.2)` validates the value, rejects it, and preserves the previous CGPA.

Q6. Why validate registered credits before earned credits?

Earned credits depend on registered credits. The constructor therefore needs the final validated registered value before deciding whether the earned-credit value is valid.

Q7. Why is sequential updating dangerous?

Separate setter calls can cause a partial update. `updateAcademicProgress()` validates the complete proposed state first and commits all three academic values together.

Q8. What is preserving a valid state?

It means an invalid operation cannot leave the object inconsistent. For example, attempting earned credits above registered credits leaves the previous earned-credit value unchanged.

Q9. Why check registered credits for zero first?

The percentage calculation divides by registered credit hours. A zero value would cause division by zero, so the function returns false before division.

Q10. Can a getter violate data hiding?

Yes. A getter returning a modifiable reference could allow outside code to change a private value without validation. Returning a value, as this program does, prevents that direct modification path.

11. Program Output Evidence

The C++ program was compiled successfully in the working environment. The transcript below records the key execution evidence.

Program :-

```
#include <iostream>
using namespace std;

class StudentRecord {
private:
    string studentName;
    string rollNumber;
    int registeredCreditHours;
    int earnedCreditHours;
    double cgpa;
    int disciplinaryCases;

public:
    // Default Constructor
    StudentRecord() {
        studentName = "Not Assigned";
        rollNumber = "Not Assigned";
        registeredCreditHours = 0;
        earnedCreditHours = 0;
        cgpa = 0.00;
        disciplinaryCases = 0;
        cout << "Default student record created." << endl;
    }

    // Parameterized Constructor
    StudentRecord(string name, string roll, int registered, int earned,
                  double studentCgpa, int cases) {
        if (name.length() >= 3)
            studentName = name;
        else
            studentName = "Not Assigned";

        if (roll.length() >= 4)
            rollNumber = roll;
        else
            rollNumber = "Not Assigned";

        if (registered >= 0 && registered <= 160)
            registeredCreditHours = registered;
        else
            registeredCreditHours = 0;

        if (earned >= 0 && earned <= 160 &&
            earned <= registeredCreditHours)
            earnedCreditHours = earned;
        else
            earnedCreditHours = 0;
```

```

    if (studentCgpa >= 0.00 && studentCgpa <= 4.00)
        cgpa = studentCgpa;
    else
        cgpa = 0.00;

    if (cases >= 0 && cases <= 3)
        disciplinaryCases = cases;
    else
        disciplinaryCases = 0;
}

// Setter Functions
bool setStudentName(string name) {
    if (name.length() < 3)
        return false;
    studentName = name;
    return true;
}

bool setRollNumber(string roll) {
    if (rollNumber != "Not Assigned")
        return false;
    if (roll.length() < 4)
        return false;
    rollNumber = roll;
    return true;
}

bool setRegisteredCreditHours(int registered) {
    if (registered < 0 || registered > 160)
        return false;
    if (registered < earnedCreditHours)
        return false;
    registeredCreditHours = registered;
    return true;
}

bool earnedCreditHours(int earned) {
    if (earned < 0 || earned > 160)
        return false;
    if (earned > registeredCreditHours)
        return false;
    earnedCreditHours = earned;
    return true;
}

bool setCgpa(double studentCgpa) {
    if (studentCgpa < 0.00 || studentCgpa > 4.00)
        return false;
    cgpa = studentCgpa;
    return true;
}

bool getDisciplinaryCases(int cases) {
    if (cases < 0 || cases > 3)
        return false;
    disciplinaryCases = cases;
}

```

```

        return true;
    }

    // Getter Functions
    string getStudentName() {
        return studentName;
    }

    string getRollNumber() {
        return rollNumber;
    }

    int setRegisteredCreditHours() {
        return registeredCreditHours;
    }

    int earnedCreditHours() {
        return earnedCreditHours;
    }

    double getCgpa() {
        return cgpa;
    }

    int getDisciplinaryCases() {
        return disciplinaryCases;
    }

    // Atomic Academic Update
    bool updateAcademicProgress(int newRegisteredCredits,
                                int newEarnedCredits,
                                double newCgpa) {
        if (newRegisteredCredits < 0 ||
            newRegisteredCredits > 160)
            return false;
        if (newEarnedCredits < 0 ||
            newEarnedCredits > 160)
            return false;
        if (newEarnedCredits > newRegisteredCredits)
            return false;
        if (newCgpa < 0.00 || newCgpa > 4.00)
            return false;

        registeredCreditHours = newRegisteredCredits;
        earnedCreditHours = newEarnedCredits;
        cgpa = newCgpa;
        return true;
    }

    // Scholarship Evaluation
    bool isEligibleForScholarship() {
        if (registeredCreditHours == 0)
            return false;

        double earnedPercentage =
            (earnedCreditHours * 100.0) /
            registeredCreditHours;
    }

```

```

        return cgpa >= 3.50 &&
               earnedCreditHours >= 30 &&
               disciplinaryCases == 0 &&
               earnedPercentage >= 75.0;
    }

    // Display Function
    void displayRecord() {
        cout << "Student Name: "
              << studentName << endl;
        cout << "Roll Number: "
              << rollNumber << endl;
        cout << "Registered Credit Hours: "
              << registeredCreditHours << endl;
        cout << "Earned Credit Hours: "
              << earnedCreditHours << endl;
        cout << "CGPA: "
              << cgpa << endl;
        cout << "Disciplinary Cases: "
              << disciplinaryCases << endl;
        cout << "Scholarship Status: "
              << (isEligibleForScholarship()
                  ? "Eligible"
                  : "Not Eligible")
              << endl;
    }
};

// Helper Function
void showResult(string testName, bool result) {
    cout << testName << ": "
          << (result ? "Accepted" : "Rejected")
          << endl;
}

int main() {
    cout << "===== " << endl;
    cout << "    UNIVERSITY OF LORALAI - ASSIGNMENT 4    " << endl;
    cout << "    STUDENT RECORD MANAGEMENT SYSTEM    " << endl;
    cout << "===== " << endl;

    // OBJECT 1
    cout << "\n[OBJECT 1] Default Constructor" << endl;
    StudentRecord student1;

    cout << "\nInitial safe values:" << endl;
    student1.displayRecord();

    cout << "\nAssigning valid values through setters:" << endl;
    showResult("Name",
              student1.setStudentName("Ali Khan"));
    showResult("Roll Number",
              student1.setRollNumber("BSCS-101"));
    showResult("Registered Credits",
              student1.setRegisteredCreditHours(100));
    showResult("Earned Credits",
              student1.setEarnedCreditHours(80));
    showResult("CGPA",

```

```

        student1.setCgpa(3.60));
showResult("Disciplinary Cases",
        student1.setDisciplinaryCases(0));

cout << "\nUpdated Object 1:" << endl;
student1.displayRecord();

// OBJECT 2
cout << "\n[OBJECT 2] Valid Parameterized Object" << endl;
StudentRecord student2(
    "Sara Ahmed",
    "BSCS-202",
    120,
    100,
    3.70,
    0
);
student2.displayRecord();

// OBJECT 3
cout << "\n[OBJECT 3] Invalid Parameterized Object" << endl;
StudentRecord student3(
    "",
    "A1",
    180,
    200,
    4.70,
    -2
);
cout << "Final safe values stored by constructor:" << endl;
student3.displayRecord();

// OBJECT 4
cout << "\n[OBJECT 4] Boundary-Value Object" << endl;
StudentRecord student4(
    "Boundary Student",
    "BSCS-404",
    160,
    160,
    4.00,
    3
);
student4.displayRecord();

// TEST CASES
cout << "\n===== " << endl;
cout << "                REQUIRED TEST CASES                " << endl;
cout << "===== " << endl;

cout << "\nTest 1 - Empty student name" << endl;
showResult("Result",
        student1.setStudentName(""));

cout << "\nTest 2 - Two-character student name" << endl;
showResult("Result",
        student1.setStudentName("A1"));

cout << "\nTest 3 - Valid name after invalid attempt" << endl;

```

```

showResult("Result",
           student1.setStudentName("Ali Khan"));

cout << "\nTest 4 - Assign valid roll number to default object" << endl;
StudentRecord testRoll;
showResult("Result",
           testRoll.setRollNumber("BSCS-505"));

cout << "\nTest 5 - Attempt to change assigned roll number" << endl;
showResult("Result",
           testRoll.setRollNumber("BSCS-999"));

cout << "\nTest 6 - CGPA = -0.01" << endl;
showResult("Result",
           student1.setCgpa(-0.01));

cout << "\nTest 7 - CGPA = 4.01" << endl;
showResult("Result",
           student1.setCgpa(4.01));

cout << "\nTest 8 - CGPA = 0.00" << endl;
showResult("Result",
           student1.setCgpa(0.00));

cout << "\nTest 9 - CGPA = 4.00" << endl;
showResult("Result",
           student1.setCgpa(4.00));

cout << "\nTest 10 - Earned credits greater than registered" << endl;
showResult("Result",
           student1.setEarnedCreditHours(150));

cout << "\nTest 11 - Reduce registered credits below earned" << endl;
student1.setEarnedCreditHours(80);
showResult("Result",
           student1.setRegisteredCreditHours(60));

cout << "\nTest 12 - Negative disciplinary cases" << endl;
showResult("Result",
           student1.setDisciplinaryCases(-1));

cout << "\nTest 13 - Scholarship with registered credits = 0" << endl;
StudentRecord zeroCredits;
cout << "Scholarship: "
     << (zeroCredits.isEligibleForScholarship()
          ? "Eligible"
          : "Not Eligible")
     << endl;

cout << "\nTest 14 - Valid atomic academic update" << endl;
showResult(
    "Result",
    student1.updateAcademicProgress(
        120,
        90,
        3.80
    )
);

```

```

student1.displayRecord();

cout << "\nTest 15 - Invalid atomic academic update" << endl;
cout << "Before update: "
    << student1.getRegisteredCreditHours()
    << ", "
    << student1.getEarnedCreditHours()
    << ", "
    << student1.getCgpa()
    << endl;

showResult(
    "Result",
    student1.updateAcademicProgress(
        120,
        125,
        3.50
    )
);

cout << "After update: "
    << student1.getRegisteredCreditHours()
    << ", "
    << student1.getEarnedCreditHours()
    << ", "
    << student1.getCgpa()
    << endl;

cout << "\n===== " << endl;
cout << "                END OF ASSIGNMENT                " << endl;
cout << "===== " << endl;

return 0;
}

```

Output :

```

=====
UNIVERSITY OF LORALAI - ASSIGNMENT 4
STUDENT RECORD MANAGEMENT SYSTEM
=====

[OBJECT 1] Default Constructor
Default student record created.

Initial safe values:
Student Name: Not Assigned
Roll Number: Not Assigned
Registered Credit Hours: 0
Earned Credit Hours: 0
CGPA: 0
Disciplinary Cases: 0
Scholarship Status: Not Eligible

Assigning valid values through setters:
Name: Accepted
Roll Number: Accepted
Registered Credits: Accepted
Earned Credits: Accepted
CGPA: Accepted
Disciplinary Cases: Accepted

Updated Object 1:
Student Name: Ali Khan
Roll Number: BSCS-101
Registered Credit Hours: 100
Earned Credit Hours: 80
CGPA: 3.6
Disciplinary Cases: 0
Scholarship Status: Eligible

[OBJECT 2] Valid Parameterized Object
Student Name: Sara Ahmed
Roll Number: BSCS-202
Registered Credit Hours: 120
Earned Credit Hours: 100
CGPA: 3.7
Disciplinary Cases: 0
Scholarship Status: Eligible

[OBJECT 3] Invalid Parameterized Object
Final safe values stored by constructor:
Student Name: Not Assigned
Roll Number: Not Assigned
Registered Credit Hours: 0
Earned Credit Hours: 0
CGPA: 0
Disciplinary Cases: 0
Scholarship Status: Not Eligible

[OBJECT 4] Boundary-Value Object
Student Name: Boundary Student
Roll Number: BSCS-404
Registered Credit Hours: 160
Earned Credit Hours: 160
CGPA: 4
Disciplinary Cases: 3
Scholarship Status: Not Eligible

=====
REQUIRED TEST CASES
=====

Test 1 - Empty student name
Result: Rejected

Test 2 - Two-character student name
Result: Rejected

Test 3 - Valid name after invalid attempt
Result: Accepted

Test 4 - Assign valid roll number to default object
Default student record created.
Result: Accepted

Test 5 - Attempt to change assigned roll number
Result: Rejected

Test 6 - CGPA = -0.01
Result: Rejected

Test 7 - CGPA = 4.01
Result: Rejected

Test 8 - CGPA = 0.00
Result: Accepted

Test 9 - CGPA = 4.00
Result: Accepted

Test 10 - Earned credits greater than registered
Result: Rejected

Test 11 - Reduce registered credits below earned
Result: Rejected

Test 12 - Negative disciplinary cases
Result: Rejected

Test 13 - Scholarship with registered credits = 0
Default student record created.
Scholarship: Not Eligible

Test 14 - Valid atomic academic update
Result: Accepted
Student Name: Ali Khan
Roll Number: BSCS-101
Registered Credit Hours: 120
Earned Credit Hours: 90
CGPA: 3.8
Disciplinary Cases: 0
Scholarship Status: Eligible

Test 15 - Invalid atomic academic update
Before update: 120, 90, 3.8
Result: Rejected
After update: 120, 90, 3.8

=====
END OF ASSIGNMENT
=====

```

12. Rubric Alignment

Rubric Area	Implemented Evidence
Class structure / private members	StudentRecord with six private data members.
Default constructor	Safe values + required message.
Parameterized constructor	All six inputs validated in logical order.
Setters / getters	All required member functions included.
Cross-field validation	Registered/earned credit relationship enforced.
Controlled roll number	Valid assignment only while Not Assigned.
Atomic update	All-or-nothing updateAcademicProgress().
Scholarship evaluation	Four conditions + zero-credit safety check.
Testing / boundary cases	Four objects + 15 compulsory tests.
Analytical answers	All 10 questions answered.
Readability / presentation	Meaningful names, comments, labels, organized report.

13. Submission Checklist

- Replace Student Name and Class ID on the report cover.
- Rename the source file using your actual Class ID: ClassID_Assignment4.cpp.
- Rename the report using your actual Class ID: ClassID_Assignment4_Report.pdf.
- Review the console output once on your own computer and add a screenshot if required.
- Submit both files through Google Classroom; the assignment states that email, WhatsApp, and hand submission are not accepted.